

Chapitre 3 – Sécurisation du code – Gestion des exceptions

I.	Objectif.....	1
II.	Des exceptions organisées en classes.....	1
III.	Mise en œuvre.....	2
1.	Utilisation classique	2
2.	Déclenchement manuel d'exception	4
IV.	Application avec EpokaAbsences.....	5
V.	Conclusion	9
VI.	PPE	10

I. Objectif

L'exécution du code peut produire des erreurs. Deux façons existent pour traiter ces erreurs :

- Un code retour (de la fonction appelée) indique si une erreur s'est produite, et donne une précision sur celle-ci (souvent un code à 0 pour indiquer aucune erreur, et de 1 à N pour signaler une anomalie) ;
- Lever une exception, si le langage supporte cette fonctionnalité évoluée ;

Le code retour est un procédé ancien, de moins en moins utilisé car rudimentaire.

A l'inverse, les exceptions produisent l'arrêt de l'exécution du code, et donnent le moyen **d'intercepter** cet arrêt.

La gestion des exceptions propose aussi d'exécuter une partie de code même après une exception. Cas typique :

1. Le code alloue des ressources ;
2. Le code utilise ces ressources ;
3. Le code libère les ressources ;

Une exception dans la partie 2 doit quand même être suivie de la partie 3.

II. Des exceptions organisées en classes

Toutes les exceptions sont dérivées d'une classe de base. En C#, c'est *Exception*.

Cela permet pour chaque exception d'ajouter des propriétés permettant de décrire l'exception.

Extrait de la documentation *Microsoft C#* :

Exception type	Base type	Description	Example
Exception	Object	Base class for all exceptions.	None (use a derived class of this exception).
SystemException	Exception	Base class for all runtime-generated errors.	None (use a derived class of this exception).
IndexOutOfRangeException	SystemException	Thrown by the runtime only when an array is indexed improperly.	Indexing an array outside its valid range : <code>arr[arr.Length+1]</code>

NullReferenceException	SystemException	Thrown by the runtime only when a null object is referenced.	<code>object o = null; o.ToString();</code>
AccessViolationException	SystemException	Thrown by the runtime only when invalid memory is accessed.	Occurs when interoperating with unmanaged code or unsafe managed code, and an invalid pointer is used.
InvalidOperationException	SystemException	Thrown by methods when in an invalid state.	Calling <code>Enumerator.GetNext()</code> after removing an Item from the underlying collection.
ArgumentException	SystemException	Base class for all argument exceptions.	None (use a derived class of this exception).
ArgumentNullException	ArgumentException	Thrown by methods that do not allow an argument to be null.	<code>String s = null; "Calculate".IndexOf (s);</code>
ArgumentOutOfRangeException	ArgumentException	Thrown by methods that verify that arguments are in a given range.	<code>String s = "string"; s.Chars[9];</code>
ExternalException	SystemException	Base class for exceptions that occur or are targeted at environments outside the runtime.	None (use a derived class of this exception).
COMException	ExternalException	Exception encapsulating COM HRESULT information.	Used in COM interop.
SEHException	ExternalException	Exception encapsulating Win32 structured exception handling information.	Used in unmanaged code interop.

III. Mise en œuvre

1. Utilisation classique

On utilise la logique suivante :

Logique à appliquer	Traduction en C#
Tenter { Code à tester }	try { Code à tester }
En cas d'erreur { Affichage / Gestion de l'erreur }	catch (Exception expt) { Affichage / Gestion de l'erreur }
Toujours finir avec { Libération de ressource(s) }	finally { Libération de ressource(s) }

Attention :

1. Seul le code présent dans *try* est « protégé ». Une erreur qui se produit dans *catch* ou bien *finally* n'est pas protégée et déclenche à son tour une exception (à éviter !).

2. La partie *catch* peut être omise. Dans ce cas, lorsque l'exception se produit, le bloc *finally* est exécuté puis l'exception remonte à un bloc *catch* puis *finally* supérieur. Exemple

Le code :

```
static void Main(string[] args)
{
    try
    {
        try
        {
            int a = 1;
            int b = 0;
            Console.WriteLine(a / b);
        }
        finally
        {
            Console.WriteLine("Fin du code");
        }
    }
    catch (Exception expt) {
        Console.WriteLine ("Une erreur s'est produite : " + expt.Message.ToString());
    }
    finally {
        Console.WriteLine("Fin de l'exécution");
    }
    Console.ReadLine();
}
```

Produit le résultat :

```
Fin du code
Une erreur s'est produite : Attempted to divide by zero.
Fin de l'exécution
```

3. Le bloc *catch* est associé à une classe d'exception. Dans l'exemple précédent, toute exception est interceptée puisque on a utilisé la classe *Exception*.

Extrait de la documentation *Microsoft C#* :

Hiérarchie d'héritage

```
System.Object
  System.Exception
    System.SystemException
      System.ArithmeticException
        System.DivideByZeroException
```

Si on utilise *ArgumentNullException* comme classe d'exception, le *catch* ne se déclenche pas, ce qui veut dire que l'exception produit une erreur non gérée :

```
Unhandled Exception: System.DivideByZeroException: Attempted to divide by zero.
   at ConsoleApplication1.Program.Main(String[] args) in d:\Utilisateurs\Francois\Documents\Visual Studio 2013\Projects\ConsoleApplication1\ConsoleApplication1\Program.cs:line 17
```

- On peut éventuellement placer plusieurs blocs *catch*, en mettant les plus « précis » d'abord ;
- Le bloc *finally* n'est pas obligatoire. Normalement, si on prend une ressource qu'il faut libérer, on construit les blocs ainsi :

- Code allouant des ressources ;
- **try**
 - {
 - Code utilisant ces ressources ;
 - }
- Eventuellement **catch**
 - {
 - ...
 - }
- **finally**
 - {
 - Code libérant les ressources ;
 - }

Exemple :

```
private void test() {
    SqlConnection connexion = new SqlConnection();
    //...
    connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("SELECT * FROM Motif", connexion);
        SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
        try
        {
            while (sqlDataReader.Read())
            {
                // Remplissage de la liste de l'onglet Salaries
                ListViewItem lvi = new ListViewItem();
                // ...
                lvSalaries.Items.Add(lvi);
            }
        }
        finally
        {
            sqlDataReader.Close();
        }
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        connexion.Close();
    }
}
```

2. Déclenchement manuel d'exception

N'importe quelle exception peut être déclenchée par le code. La commande est *throw* suivie du constructeur de l'exception. Le code suivant *throw* n'est donc pas exécuté.

Exemple :

```
private String Coller(String a, String b)
{
    if (a==null | b==null)
    {
        throw new System.ArgumentException("Au moins un des paramètres est nul", "a ou b");
    }
    return a + b;
}
```

IV. Application avec EpokaAbsences

La méthode *majListeSalaries* doit être sécurisée comme indiqué ci-dessus :


```

private void majListeSalaries()
{
    // Effacer la liste de l'onglet Salaries
    lvSalaries.Items.Clear();
    // Effectuer la dé-selection, non produite automatiquement
    lvSalaries_SelectedIndexChanged(null, null);

    // Effacer aussi la liste déroulante des salariés de l'onglet Absences
    cbSalarie.Items.Clear();
    // Effectuer la dé-selection, non produite automatiquement
    cbSalarie_SelectedIndexChanged(null, null);

    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV12; "
        + "Initial Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("SELECT * FROM Salarie", connexion);
        SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
        try
        {
            while (sqlDataReader.Read())
            {
                // Remplissage de la liste de l'onglet Salaries
                ListViewItem lvi = new ListViewItem();
                int salNo = (int)sqlDataReader["SalNo"];
                lvi.Text = salNo.ToString();
                lvi.SubItems.Add((String)sqlDataReader["SalNom"]);
                lvi.SubItems.Add((String)sqlDataReader["SalPrenom"]);
                lvSalaries.Items.Add(lvi);

                // Remplissage de la liste déroulante de l'onglet Absences
                ComboBoxItem cbi = new ComboBoxItem();
                cbi.code = salNo.ToString();
                cbi.libelle = (String)sqlDataReader["SalNom"] + " " + (String)sqlDataReader["SalPrenom"];
                cbSalarie.Items.Add(cbi);
            }
        }
        finally
        {
            sqlDataReader.Close();
        }
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        connexion.Close();
    }
}

```

Le clic sur le bouton *bAjouterSalarie* doit être sécurisé comme indiqué ci-dessus :


```

private void bAjouterSalarie_Click(object sender, EventArgs e)
{
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV12; " +
        "Initial Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("INSERT INTO Salarie (SalNom,SalPrenom) VALUES ('"
            + tbNom.Text + "', '" + tbPrenom.Text + "')", connexion);
        sqlCmd.ExecuteNonQuery();
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        connexion.Close();
    }
    majListeSalaries();
}

```

Il en est de même pour le bouton *bModifierSalarie* :

```

private void bModifierSalarie_Click(object sender, EventArgs e)
{
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV12; " +
        "Initial Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("UPDATE Salarie SET SalNom='" + tbNom.Text
            + "', SalPrenom='" + tbPrenom.Text + "' WHERE SalNo="
            + lvSalaries.SelectedItems[0].Text, connexion);
        sqlCmd.ExecuteNonQuery();
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        connexion.Close();
    }
    majListeSalaries();
}

```

Ainsi que pour *bSupprimerSalarie* :


```

private void bSupprimerSalarie_Click(object sender, EventArgs e)
{
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV12; " +
        "Initial Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("DELETE FROM Salarie WHERE SalNo="
            + lvSalaries.SelectedItems[0].Text, connexion);
        sqlCmd.ExecuteNonQuery();
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        connexion.Close();
    }
    majListeSalaries();
}

```

La méthode *majListeAbsences* est modifiée :

```

private void majListeAbsences()
{
    lvAbsences.Items.Clear(); // Effacer la liste de l'onglet Absences

    // Remplir les ListViewGroup avec les motifs
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV12; "
        + "Initial Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("SELECT * FROM Motif", connexion);
        SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
        try
        {
            while (sqlDataReader.Read())
            {
                String motLibelle = (String)sqlDataReader["MotLibelle"];
                ListViewGroup lvg = new ListViewGroup(motLibelle);
                lvAbsences.Groups.Add(lvg);
            }
        }
        finally
        {
            sqlDataReader.Close();
        }
    }
}

```



```

if (cbSalarie.SelectedItem != null)
{
    // Remplir les ListViewItem avec les absences du salarié,
    // en les rattachant au bon ListViewGroup (motif)
    sqlCmd.CommandText = "SELECT * FROM Absence,Motif WHERE AbsNoMotif=MotNo "
        + "AND AbsNoSalarie=" + (cbSalarie.SelectedItem as ComboBoxItem).code.ToString();
    sqlDataReader = sqlCmd.ExecuteReader();
    try
    {
        while (sqlDataReader.Read())
        {
            ListViewItem lvi = new ListViewItem();
            DateTime debut = (DateTime)sqlDataReader["AbsDebut"];
            lvi.Text = debut.ToString("ddd dd/MM/yyyy");
            DateTime fin = (DateTime)sqlDataReader["AbsFin"];
            lvi.SubItems.Add(fin.ToString("ddd dd/MM/yyyy"));
            TimeSpan ts = fin - debut;
            lvi.SubItems.Add((ts.Days + 1).ToString() + " jour(s)");
            // Rechercher le groupe :
            foreach (ListViewGroup lvg in lvAbsences.Groups)
            {
                if (lvg.Header.ToString() == (String)sqlDataReader["MotLibelle"])
                {
                    lvi.Group = lvg;
                }
            };
            lvAbsences.Items.Add(lvi);
        }
    }
    finally
    {
        sqlDataReader.Close();
    }
}
catch (SqlException sqlEx)
{
    Console.WriteLine("Erreur : " + sqlEx.Message);
}
finally
{
    connexion.Close();
}
}

```

V. Conclusion

Un code un peu plus long, mais :

- Le code est plus robuste (fiable) ;
- Sécurité dans le code (erreur SQL interceptée) ;

Toujours pas traité :

- La maintenance / évolution n'est pas facilitée ;
- Absence de sécurité dans les requêtes SQL (toute requête peut être passée, l'injection SQL est possible, ...) ;
- La migration vers un autre support de stockage (fichiers XML, autre SGBD, ...) n'est pas simple ;

VI. PPE

Vous êtes chargé de réaliser la phase 2 en sécurisant le code de la même manière avec les exceptions.

Prenez le projet de ce chapitre, et complétez-le avec le code déjà réalisé du chapitre précédent, que vous adapterez.

Gardez bien des projets distincts pour chaque chapitre.